



第三次培训讲解文稿

主题：Python 面向对象与库简介。



培训目标

（开场白）

大家好，欢迎来到我们的第三次 Python 培训。

这次课程的重点是——**面向对象编程（OOP）思想** 和 **Python 的库**。

今天我们要做到以下几点：

1. 理解面向对象编程的核心思想；
2. 能够在 Python 中定义类、创建对象；
3. 了解封装、继承、多态三大特性；
4. 知道 Python 的标准库和第三方库的作用；
5. 最后，我们将基于 OOP 优化上次写过的成绩管理系统。

回顾上次的成绩管理系统（过程式）

上次我们写过一个非常简单的成绩计算程序。

它通过输入学生姓名和成绩，直接计算平均分。

这个程序的特点是**按顺序执行**，所有逻辑都堆在一起。

但当我们处理多个学生、或者增加功能，比如加分、计算绩点等，就会变得非常麻烦。



面向过程的问题

这里我们总结一下面向过程的问题：

- 1. 数据（如成绩）和逻辑是分开的；
- 2. 程序扩展性差，修改一处容易牵连很多地方；
- 3. 难以复用，写一个新学生就要重新复制一遍逻辑。

所以我们希望——让“学生”成为一个整体。
既有自己的数据，也有计算自己成绩的能力。

二、引入面向对象思想（OOP）

过渡语句：

为了解决这些问题，我们引入“面向对象”的思维。

核心思想是：

一切皆对象 —— 用对象去抽象现实世界。

比如：学生、成绩单、教室，都可以被看作对象。
对象内部包含数据（属性）和行为（方法）。

思路	组织方式	优点
面向过程	按步骤执行	简单直接
面向对象	按对象组织	易扩展、可复用

类与对象示意图

展示图：image\第三次培训\类与对象.svg

这一页的图说明了 OOP 的核心结构：

- **类（Class）** 是模板，描述对象的共性；
- **对象（Object）** 是实例，代表具体个体；
- 每个对象都有自己的属性，比如 Alice 和 Bob 的成绩不同；
- 但它们都来自同一个类 Student。

所以：类定义了“**是什么**”，对象体现了“**具体是谁**”。

Python 中的类与对象

在 Python 中，我们用 `class` 关键字定义类。

在这里，`__init__()` 是初始化方法，当创建对象时会自动执行。

`self` 代表当前对象自身。

我们在类中定义的函数叫**方法**，用于操作类中的数据。

创建对象

当我们写 `alice = Student("Alice", [80, 90, 70])` 时，

Python 会创建一个新的 `Student` 对象，并调用 `__init__`。

这时 `alice.name` 是 `"Alice"`，`alice.scores` 是 `[80, 90, 70]`。

调用 `alice.average()`，Python 会自动把 `alice` 自身传进去。

这样每个对象就能“管理自己的数据”。

一句话总结：**类是模板，对象是具体的实例。**

四、OOP 版成绩管理系统

这页展示的是一个完整的 OOP 实现。

我们定义了：

- `average()`：计算平均分；
- `add_bonus()`：对成绩加分；
- `report()`：打印成绩报告。

这比上次的代码更清晰、更模块化，也更易维护。

示例运行

我们创建了两个学生对象：Alice 和 Bob。
每个对象都能调用自己的方法，计算平均分或加分。

你可以看到，类让我们的逻辑非常自然：

每个学生自己“知道”如何计算成绩。

这正是封装思想的体现。

类与对象关系图

重新展示 类与对象.svg

这里再强调一次：

- Student 是类（蓝图）
- alice、bob 是实例
- 每个对象都有独立的数据空间
- 对象可以调用类中定义的方法

通过图像，大家能更直观地理解“类是模板，对象是实例”的概念。

五、OOP 三大特性

讲解：

- **封装**：将数据与操作绑定在一起；
- **继承**：新类可以复用父类代码；
- **多态**：同一个方法，不同对象有不同表现。

比如：

- report() 在 GraduateStudent 中可以打印论文成绩；
- is_pass() 可以在不同类型学生中定义不同标准。

今天我们重点理解 **封装**，
继承和多态在后续讲 C++ 时会深入。

六、对比总结

这一页是对比图：

可以直接总结：

方面	面向过程	面向对象
数据	独立变量	对象属性
函数	全局函数	类方法
扩展	修改逻辑	新建子类
可读性	难维护	清晰结构

面向对象更像是在搭积木，灵活、清晰、可扩展。

七、Python 标准库简介

Python 的标准库非常强大，基本功能都不用重写。

举例：

- `os` 用于操作文件；
- `datetime` 用于时间；
- `math`、`random` 用于数学计算；
- `csv`、`json` 读写文件；
- `collections` 提供更强的数据结构。

这就是 Python “电池全满” 的理念。

示例： `datetime`

这里的代码示例展示如何获取当前时间并格式化输出。

这类操作在日志、文件命名、统计报告中非常常见。

八、第三方库简介

第三方库由社区维护，用 `pip install` 安装。

举例：

- 数据分析： `pandas` , `numpy`
- 可视化： `matplotlib` , `seaborn`
- AI： `pytorch` , `tensorflow`
- Web 开发： `flask` , `django`
- 游戏开发： `pygame`

Python 的强大就在于生态丰富。几乎所有问题，都有现成的库。

任务描述（Task）

接下来我们设计一个小任务：

- 输入成绩 CSV 文件；
- 计算加权平均绩点与加权平均成绩。

这部分可以帮助大家体会到库的便利性。

不需要循环、不需要复杂逻辑，用几行 `pandas` 就搞定。

使用的库与方法

这里我们重点介绍 `pandas` 。

它是 Python 最重要的数据分析库，能像 Excel 一样处理表格。

优点：

- 操作简单：几行代码搞定复杂统计；
- 性能高：底层基于 C 实现；
- 支持多种文件格式；
- 广泛用于数据科学和 AI。

示例：使用 pandas 计算加权平均

这段代码演示如何读取 CSV、计算总学分和加权平均。

你可以强调：

- 向量化计算避免循环；
- 代码简洁；
- 可拓展性强（能按学期、课程分类统计）。

输出示例

这里展示了计算结果：

- 总学分；
- 平均绩点；
- 平均成绩；
- 以及每门课对加权的贡献。

可以引导学生思考：

- 如何只统计某个学期？
- 如何筛选非公选课？
- 如何加入异常数据处理？

这些问题可以作为进阶思考。

九、总结与展望

我们这次课的核心收获：

收获	内容
OOP 思想	类与对象
三大特性	封装、继承、多态

收获	内容
标准库	常用功能一览
第三方库	pandas 实践

下次我们会进入更底层的 **C 语言基础与面向对象思想的前身**。

课后练习

建议大家尝试：

1. 给 Student 类增加功能，比如最高分、是否及格；
2. 设计一个 Classroom 类，能存多个学生；
3. 用 json 库保存数据。

这些练习能帮助你真正掌握面向对象的思维。

结语

感谢大家的参与！

如果遇到任何问题，欢迎随时在群里交流。

希望今天的内容能帮助大家写出更结构化、更优雅的 Python 程序。